

Contact: David Zhang – dzhang [at] cs [dot] toronto [dot] edu

All questions are taken from David Karger's 6.854 Advanced Algorithms Homework ([link](#)):

1. You have reached a river (modeled as a straight line) and must find a bridge to cross it. The bridge is at some integer coordinate upstream or downstream

- (a) Give a 9-competitive deterministic algorithm for optimizing the total distance travelled up and downstream before you find the bridge. This is optimal for deterministic strategies.

The algorithm is as follows:

- Set the initial direction upstream;
- Start from $i = 0$. Then, execute *step i*. If we don't find the bridge then increment i and repeat.

On *step i*, travel 2^i units in the current direction and terminate immediately if we reach the bridge. Otherwise flip around and travel 2^i units in the opposite direction to return to the origin. In each step, we travel 2^{i+1} units.

Suppose that $2^i < |b| \leq 2^{i+1}$. In the worst case, we fall just short of discovering the bridge at position b – that is, step i explores in the same direction of b . To reach b , we need to travel twice the distance on the other side before returning to the origin and finding b in the next step. Hence, the distance that we travel following step i is at most $5b$. Also note that $2^i < |b|$ so $i < \lg b$, i.e. we take at most $\lg b$ steps to almost reach b . Hence, the maximum distance that we will ever travel is bounded by $5b + \sum_{j=0}^{\lg b} 2^{j+1} = 5b + 2(2^{\lg b+1} - 1) < 9b$, yielding the desired ratio.

- (b) Give a randomized 7-competitive algorithm for the problem.

Our algorithm suffers if we choose the wrong initial direction. Remedy this by flipping a fair coin to decide whether to explore up or downstream first. Suppose that $2^i < |b| \leq 2^{i+1}$. In the worst case, step i explores in the same direction of b and we remain 9-competitive. If we choose the right direction, then step i explores in the opposite direction and we will reach b during step $i + 1$. Hence, the total distance that we travel is bounded by $b + \sum_{j=0}^{\lg b} 2^{j+1} = b + 2(2^{\lg b+1} - 1) < 5b$. We traverse at most $5b/2 + 9b/2 = 7b$ units in expectation, so this randomized algorithm is 7-competitive.

2. Consider an online version of the Knapsack problem where you get items one at a time and have to decide whether to put them into your Knapsack. If you run out of space you need to discard something and can never get it back later. What sort of competitive ratio can you achieve versus the offline (optimal) knapsack? WLOG, assume the knapsack has size 1 and each item has size no greater. Consider the following two algorithms: MAX, which simply keeps the highest value item, and GREEDY, which gives each item a *density* equal to its value divided by its size, and keeps as many of the highest density items as possible.

- (a) Prove that neither GREEDY nor MAX is competitive.

-
- MAX: Send $1/\epsilon^2$ items, each with $s_i = \epsilon^2$ and $v_i = \epsilon$, followed by a single item of unit size and value. MAX will pack the unit item while an optimal knapsack packs the $1/\epsilon^2$ items to achieve $1/\epsilon$ profit. Decreasing ϵ will make the ratio arbitrarily large.
 - GREEDY: Send a single job with unit size and value, followed by a job with ϵ^2 size and ϵ value. A unit capacity knapsack can only accept one of the two jobs – GREEDY accepts the one with ϵ value (having density $1/\epsilon$) while an optimal knapsack accepts the item with unit value. Again, decreasing ϵ makes the ratio arbitrarily bad.

- (b) Consider *fractional GREEDY* which is able to discard or keep arbitrary *fractions* of items (of high density). Prove that it does at least as well as the (integer) optimum.

In the fractional setting, if the total weight of items that OPT packs is less than 1, then the total weight across all items must also be less than 1. In this case, our algorithm would just need to accept every item whole to be optimal. Thus, it is more interesting to consider when the sum of all item weights is at least 1. In this case, OPT will maximize the sum of densities of its accepted items, which will just be the total value that it packs since its final weight will be 1.

To maximize density, accept the densest items until the packed weight exceeds 1. Then, remove fractions (or whole amounts) of the least dense item(s) to keep the total weight at 1. Fractional GREEDY can do this in an online fashion by removing fractional (or whole) amounts of items with the smallest density, whenever it encounters an item whose density is greater than the minimum density in the set. Hence, the solution that fractional GREEDY produces is identical to the optimal offline solution, which is at least as good as the integer OPT since the fractional variant is a relaxation of it.

- (c) Prove that randomly choosing between GREEDY and MAX is a 2-competitive strategy. **Hint:** compare to fractional GREEDY. Which will have how many fractional items?

Note that GREEDY and fractional GREEDY will accept the same items, but GREEDY will completely discard the least dense item while fractional GREEDY will keep a fraction of it. Further, note that the value provided by this least dense item is at most the value of the item that MAX accepts, i.e. it is bounded by the maximum value. Hence, the sum of GREEDY's knapsack value plus MAX's knapsack value is greater than the total value that OPT packs. Since we choose each strategy with $1/2$ probability, it follows that we capture at least half of the optimal knapsack's value in expectation.

3. Consider a process waiting to acquire a lock *held* by a different process in a multiprocessing system. There are two available options for the process. The process can either *spin*, i.e., continuously try to acquire the lock in a loop, at a cost proportional to the length of the time that it spins¹ or it can **block**, temporarily shutting off to let some other process use the processor. Blocking has a large cost $B > 0$ reflecting the cost of restarting the process and restoring its state (i.e., context-switch cost). For example, a process that spins for t seconds and then blocks will incur a cost of $t + B$.

Spinning prevents other processes that are waiting from getting a processor; however, spinning might be useful since the lock may become available immediately. Your goal is to devise a minimum-cost strategy for deciding how long a process should spin before it blocks.

The challenge is that the optimal time to block depends on how long it will be before the lock is free, and this information is not available in an online setting. In this problem, we will devise an online randomized algorithm with competitive ratio $e/(e - 1) \approx 1.58$.

- (a) It should be clear that any (randomized) algorithm A is defined entirely by the probability density function $\pi(t)$ of the time at which the process blocks. Let σ_τ denote the scenario where the lock is held for time τ . Compute the expected cost of algorithm A given σ_τ , $\mathbb{E}[C_A(\sigma_\tau)]$.

Suppose that A blocks after $t < \tau$ seconds. The actual cost that A incurs is $t + B$ and the expected cost is $\int_0^\tau (t + B)\pi(t)dt$. On the other hand, suppose that A never blocks – that is, it spins until time τ when the lock releases. The actual cost that this incurs is τ and therefore the expected cost is $\tau \int_\tau^\infty \pi(t)dt$. It follows that the overall expected cost of A is

$$\mathbb{E}[C_A(\sigma_\tau)] = \int_0^\tau (t + B)\pi(t)dt + \tau \int_\tau^\infty \pi(t)dt$$

- (b) Consider the optimal off-line algorithm A^* for this case (equipped with knowledge of τ) and compute an exact expression for the cost of $\mathbb{E}[C_{A^*}(\sigma_\tau)]$. (This may depend on B).

A^* sleeps immediately if the lock releases after B or more seconds. Otherwise if $\tau < B$, A^* will simply sleep for τ seconds. A^* is optimal (and deterministic) because it never pays for more than it needs to. $\mathbb{E}[C_{A^*}(\sigma_\tau)] = B$ if $\tau \geq B$; otherwise $\mathbb{E}[C_{A^*}(\sigma_\tau)] = \tau$.

- (c) Let $\alpha > 0$ and consider $\mathbb{E}[C_A(\sigma_\tau)]$. In light of the previous part, express the conditions on $\pi(t)$ so that $\mathbb{E}[C_A(\sigma_\tau)]$ is $(1 + \alpha)$ competitive with the optimal offline cost. **Hint:** this part should follow immediately from parts (a) and (b).

$\pi(t)$ must be given subject to $\mathbb{E}[C_A(\sigma_\tau)] = \int_0^\tau (t + B)\pi(t)dt + \tau \int_\tau^\infty \pi(t)dt \leq (1 + \alpha)B$ if $\tau \geq B$, and $\mathbb{E}[C_A(\sigma_\tau)] = \int_0^\tau (t + B)\pi(t)dt + \tau \int_\tau^\infty \pi(t)dt \leq (1 + \alpha)\tau$ if $\tau < B$.

¹For simplicity, we will assume that the cost in this case is indeed equal to the time.

- (d) Derive the expression for $\pi^*(\cdot)$ defined to be the function that minimizes α from the previous part. **Hint:** set the inequalities to equalities and differentiate (possibly more than once) with respect to τ .

We devise a density function that minimizes α . First, replace the inequalities with equalities and differentiate with respect to τ .

Begin by taking the derivative of A 's expected cost

$$\begin{aligned} \frac{d}{d\tau} \left(\int_0^\tau (t+B)\pi(t)dt + \tau \int_\tau^\infty \pi(t)dt \right) &= (\tau+B)\pi(\tau) + \int_\tau^\infty \pi(t)dt - \tau\pi(\tau) \\ &= B\pi(\tau) + \int_\tau^\infty \pi(t)dt \end{aligned}$$

If $\tau \geq B$, then A^* 's differentiated cost is $\frac{d}{d\tau}(1+\alpha)B = 0$ and $B\pi(\tau) + \int_\tau^\infty \pi(t)dt = 0$. Therefore, $\pi(\cdot)$ cannot be positive past B . Since $\pi(\cdot)$ is a probability density function, it must be non-negative. Hence, it must evaluate to 0 at inputs of B or larger.

Next, suppose that $\tau < B$. A^* 's differentiated cost is $\frac{d}{d\tau}(1+\alpha)\tau = 1+\alpha$. By rearranging terms,

$$\alpha = B\pi(\tau) + \int_\tau^\infty \pi(t)dt - 1$$

Differentiating on τ once more reveals that $B\pi'(\tau) - \pi(\tau) = 0$, so $\pi'(\tau) = \pi(\tau)/B$. Therefore, $\pi(\tau)$ must be some constant multiple, say C , of $e^{t/B}$. Combining both cases yields $\pi^*(\tau) = Ce^{t/B}$ if $\tau < B$ and $\pi^*(\tau) = 0$ if $\tau \geq B$. Since $\pi^*(\cdot)$ is a probability density function, it must satisfy $\int_0^\infty \pi^*(t)dt = \int_0^B Ce^{t/B}dt = \int_0^B Ce^{t/B}dt = 1$. Hence, $C = \frac{1}{B(e-1)}$ and

$$\pi^*(\tau) = \begin{cases} \frac{e^{t/B}}{B(e-1)} & \tau < B \\ 0 & \tau \geq B \end{cases}$$

- (e) Conclude that algorithm A achieves a competitive ratio of $e/(e-1) \approx 1.58$

$\pi^*(\cdot)$ guarantees that A only blocks prior to B , so $\alpha = B\pi(\tau) + \int_\tau^\infty \pi(t)dt - 1$. Notice that

$$\begin{aligned} B\pi^*(\tau) + \int_\tau^\infty \pi^*(t)dt - 1 &= B \left(\frac{e^{\tau/B}}{B(e-1)} \right) + \int_\tau^B \frac{e^{t/B}}{B(e-1)}dt - 1 \\ &= \frac{e^{\tau/B}}{(e-1)} + \frac{e^{B/B}}{e-1} - \frac{e^{\tau/B}}{e-1} - 1 \\ &= e/(e-1) - 1 \end{aligned}$$

A is $(1+\alpha)$ -competitive, so the desired ratio follows immediately.

4. Consider a collection of faculty available to give you advice about your future path in life. Since they are busy, each only answers yes or no questions. Some are wiser than others, and are more likely to be correct in their advice. You would like, looking back, to have taken the advice of the wisest faculty member, but ahead of time you don't know who that is. Consider the following online algorithm: each time you ask a question, you ask all the faculty, and go with a "weighted majority" opinion as follows:

1. Initially, each faculty member has a weight of 1.
2. After asking a question, take the (yes/no) answer of larger total weight.
3. Upon discovering which answer is correct, halve the weight of each faculty member who answered incorrectly.

You will show that, using this algorithm, you make at most $2.41(m + \lg n)$ mistakes, where m is the number of mistakes made by the wisest faculty, and n is the number of faculty. (Thus, in the language of asymptotic competitive ratios, you are 2.41-competitive against asking the best faculty member.)

- (a) Prove that, when the online algorithm makes a mistake, the total weight of the faculty decreases by a factor of $4/3$. Use this to upper bound the total weight assigned to faculty.

Suppose that the total weight of faculty before asking the next question is W . Further, suppose that the weighted majority makes a mistake on the next question. The weight of the majority is at least $W/2$ so we lose at least $W/4$. Hence, we're left with at most $3W/4$. If the weighted majority makes k mistakes in total, then the total weight of faculty after answering the last question is at most $(3/4)^k n$.

- (b) Lower-bound the weight assigned to faculty by considering the weight of the wisest faculty member in terms of m .

The wisest faculty member makes m mistakes. If they are in the weighted majority when they make each of those mistakes, then their final weight will be $(1/2)^m$. Otherwise their weight does not decrease after reweighting. Hence, the wisest faculty's weight after answering every question is at least $(1/2)^m$.

- (c) Combine the above two parts to prove the claim.

The final total weight of faculty is a trivial bound on the final weight of the wisest faculty, i.e. $(1/2)^m \leq (3/4)^k n$. All that remains is to bound k , the total number of mistakes that the weighted majority makes. Taking logs,

$$\begin{aligned} -m = \log_2(1/2)^m &\leq \log_2((3/4)^k n) = k \log_2(3/4) + \log_2 n \\ k \log_2(4/3) &\leq m + \log_2 n \\ k &\leq 2.41(m + \log_2 n) \end{aligned}$$

Each inequality is a rearrangement of the previous one.

With randomization, you can do better. We modify the online algorithm as follows:

1. Instead of going with the majority, choose one faculty member with probability proportional to their weight, and follow their opinion.
 2. Instead of multiplying incorrect faculty weights by $1/2$, multiply them by some $\beta < 1$ to be determined later.
- (d) For a given question, let F denote the fraction of the weight of faculty with the wrong answer. Give an expression for the multiplicative change in the total weight as a result of reweighting for this question.

Suppose that the total weight of faculty was W prior to reweighting. After reweighting, the total weight will be $F\beta W + (1 - F)W = (F\beta - F + 1)W$ so the multiplicative change is $F\beta - F + 1$.

- (e) Arguing much as in the deterministic case, prove that the (expected) number of wrong answers will be at most

$$\frac{m \ln(1/\beta) + \ln n}{1 - \beta}$$

so for example, setting $\beta = 1/2$ gives $1.39m + 2 \ln n$, for a competitive ratio of 1.39.

Suppose that there are T questions. Let F_i denote the fraction of the weight of faculty with the wrong answer before the algorithm answers question $i \in \{1, \dots, T\}$. Note that the total weight of faculty after answering every question is $n \prod_{i=1}^T (F_i \beta - F_i + 1)$. We obtain $1 - F_i(1 - \beta) \leq e^{-F_i(1-\beta)}$ after rearranging the multiplicative factor and applying an exponential inequality. Hence,

$$n \prod_{i=1}^T (1 - F_i(1 - \beta)) \leq n e^{-(1-\beta) \sum_{i=1}^T F_i}$$

The number of mistakes that our algorithm makes is the sum over all questions of the probability of choosing a wrong faculty member. That is, we make $\sum_{i=1}^T F_i$ mistakes in expectation. The wisest faculty's final weight is at least β^m , so much like the deterministic case, we have $\beta^m \leq n e^{-(1-\beta) \sum_{i=1}^T F_i}$. Taking the natural logarithm of both sides and rearranging yields

$$\sum_{i=1}^T F_i \leq \frac{m \ln(1/\beta) + \ln n}{1 - \beta}$$

- (f) Show that if one uses the “right” β , one can limit the number of errors to $m + \ln n + O(\sqrt{m \ln n})$, achieving a 1-competitive algorithm (in the asymptotic sense). (This β can actually be found online by “repeated doubling” as the mistakes build up.)

Suppose that $1/\beta = 1 + x$. Then, $1 - \beta = x/(1 + x)$ and the bound simplifies to

$$\begin{aligned} \frac{m \ln(1 + x) + \ln n}{x/(1 + x)} &= (1 + x) \frac{m \ln(1 + x) + \ln n}{x} \\ &\leq (1 + x) \left(m + \frac{\ln n}{x} \right) \quad \text{since } \ln(1 + x) \leq x \\ &= m + \ln n + xm + \frac{\ln n}{x} \end{aligned}$$

Setting $x = \sqrt{\frac{\ln n}{m}}$ yields $m + \ln n + 2\sqrt{m \ln n}$. Hence, setting $\beta = \frac{1}{1 + \sqrt{\frac{\ln n}{m}}} = \frac{\sqrt{m}}{\sqrt{m} + \sqrt{\ln n}}$ will achieve the desired error bound.